Week 10 — Functions: Math Machines

Name: ___ Date: ___

> *`def` **builds** the machine · calling **runs** it · `return` **hands the answer back**. Math's f(3) and Python's f(3) are the same thing.*

Today's words

Word	What it means
function	a machine: input in the slot, output out the chute
`def`	builds a machine (building it doesn't run it!)
parameter	the input slot's name — the x in `def double(x):`
call	using the machine: `double(5)` — parentheses press the button
`return`	hands the answer back so the program can keep using it

1 · Be the machine 🤖

Fill each table by running the machine in your head.

```
def triple(x):                    def machine(x):
    return x * 3                      return x * x - 1
```

in (x)	`triple(x)`		in (x)	`machine(x)`
1			2	
2			3	
5			4	
10			10	

2 · Match the call to its output 🔌

Draw a line from each call to what it prints. One output is left over!

```
def double(x):                    def gap(a, b):
    return x * 2                      return a - b
```

Call		Output
`print(double(6))`		`-5`
`print(gap(9, 4))`		`8`
`print(gap(4, 9))`		`12`
`print(double(double(2)))`		`5`
		`24`

3 · Build a machine on paper 🔧

Write a complete `def` for a machine named `add_ten` that takes one number and hands back that number plus 10. (Two lines — don't forget the colon, the indent, and the keyword that hands the answer back.)

🧠 Brain teaser (optional — take it home)

Two machines: `def double(x): return x * 2` and `def add_three(x): return x + 3`

Work **inside out**, like nested parentheses:

1. `double(add_three(2))` = _____
2. `add_three(double(2))` = _____
3. Same machines, same input, both orders — same answer? Circle: **YES / NO**

In math language that's f(g(2)) vs g(f(2)). Bring your answers next week for a shout-out — and find out what this has to do with putting on socks and shoes.